

QS-Prozesse – Auflösung nach den W-Fragen

Jede Qualitätsmaßnahme wird nach denselben sechs Fragen beschrieben. Die Systematik stellt sicher, dass zu jeder Maßnahme Zweck, Zuständigkeit, Auslöser, Ergebnis und Ablauf festgelegt sind und begründet ist, warum die Maßnahme für das jeweilige Qualitätsziel ausreicht.

W	Frage	Bedeutung
Warum	Welches Problem oder Risiko wird adressiert?	Zweck der Maßnahme
Wer	Wer führt durch, wer prüft, wer gibt frei?	Zuständigkeit
Wann	Welches Ereignis löst die Maßnahme aus?	Zeitpunkt oder Rhythmus
Was	Welche Bedingung muss erfüllt sein und welcher Nachweis entsteht?	Freigabekriterium und Artefakt
Wie	Wie läuft die Prüfung konkret ab?	Ablauf
Gut genug weil	Warum reicht diese Maßnahme für das Qualitätsziel aus?	Wirksamkeit und Grenzen

Begriffe

- p95 – der Wert, den 95 von 100 Anfragen unterschreiten. Gemessen wird damit nicht der Durchschnitt, sondern wie langsam es in den schlechtesten Fällen wird.
- S1 bis S4 – Schweregrade für Fehler. Die Definitionen und die jeweils festgelegte Reaktion stehen in P14.

P1 – Definition of Done

Warum

- Unterschiedliches Verständnis von „fertig“ führt zu Konflikten
- Nacharbeit wird vermieden
- Die DoD ist Voraussetzung für alle folgenden Prozesse
- Verweist auf P2, P3, P4 und P12 – an einer Stelle wird sichtbar, ob alle Schritte durchlaufen wurden

Wer

- Alle Teammitglieder erfüllen die DoD
- QS-Beauftragte prüfen die DoD in periodischen Abständen

Wann

- Auslöser: eine User Story soll als fertig gelten
- Prüfung beim Statuswechsel der Story auf „fertig“
- Die DoD selbst wird einmalig im Team beschlossen und im Repository abgelegt

Was

- Freigabe: alle Punkte erfüllt – ohne vollständige DoD gilt die Story nicht als fertig
- DoD-Checkliste im Repository als verbindliche Vorlage
- Abgehakte DoD im Issue als Nachweis pro Story
- Bei nicht erfüllter DoD bleibt die Story in Bearbeitung

Wie

- Sechs Punkte je Story: Akzeptanzkriterien mit Test, Review bestanden (P2), Pipeline erfolgreich und Coverage eingehalten (P3, P4), Dokumentation angepasst (P12), in Staging funktionsfähig, kein offener Fehler S1 oder S2
- Die DoD liegt im Repository und wird in jedem Issue referenziert, das Abhaken erfolgt im Issue

Gut genug als Qualitätsmaßnahme weil:

- Macht implizite Erwartungen explizit und damit prüfbar – „fertig“ ist nicht mehr Auslegungssache
- Braucht keine zusätzliche Software, nur eine Checkliste, und gilt ohne Ausnahme für jede Story
- Die abgehakte DoD im Issue ist ein dauerhafter Nachweis und macht die Maßnahme messbar
- Grenze: geprüft wird, dass ein Punkt erledigt wurde, nicht wie gut – die Tiefe liefern erst P2 bis P12
- Grenze: beruht auf Selbstauskunft und wirkt nur mit Gegenprüfung

P2 – Pull Request und Review

Warum

- Qualitätsziel: Weiterentwicklungseignung
- Fehler werden gefunden, bevor sie in dev oder main landen
- Ohne Review wird der Aufbau des Codes mit der Zeit unsauber – einzelne Abweichungen fallen kaum auf, in Summe wird der Code schwer änderbar
- Mindestens zwei Personen kennen jede Änderung
- Versehentlich im Code stehende Passwörter oder Schlüssel werden vor dem Merge entdeckt – danach bleiben sie dauerhaft in der Git-Historie

Wer

- Der Autor der Änderung eröffnet den Pull Request
- Review und Freigabe durch den Teamlead
- Bei Änderungen an der OpenAPI-Schnittstelle zusätzlich Review durch das andere Team (Frontend beziehungsweise Backend)

Wann

- Auslöser: eine Änderung soll nach dev oder main
- Vor jedem Merge, ausnahmslos – direkte Commits auf dev und main sind ausgeschlossen

Was

- Freigabe: Freigabe durch den Teamlead und vollständig erfolgreiche Pipeline
- Nachweis: Historie des Pull Requests mit Freigabe, CI-Status am Commit
- Ohne Freigabe kein Merge

Wie

- Branch von dev abzweigen, keine direkten Commits auf dev oder main
- Lint, Typecheck und Tests lokal ausführen
- Pull Request mit Template eröffnen: Zweck, betroffene Komponenten, durchgeführte Tests, Auswirkung auf OpenAPI
- Die CI führt P3 und P4 aus

- Im Review wird geprüft: zu jedem Akzeptanzkriterium existiert ein Test, Änderungen an bestehenden Tests sind begründet, es gibt keine unbestimmten Typen (any) ohne Begründung, die Programmlogik liegt in der Service- und nicht in der Router-Schicht, Schichten- und Ordnerstruktur sind eingehalten, die OpenAPI-Beschreibung passt zum Code, die Dokumentation ist nachgezogen, es stehen keine Zugangsdaten im Code
- Nach erhaltener Freigabe mergt der Autor und löscht den Branch

Gut genug als Qualitätsmaßnahme weil:

- Jede Änderung wird von einer zweiten Person geprüft, bevor sie in den gemeinsamen Stand gelangt
- Das zusätzliche Review greift nur bei OpenAPI-Änderungen, also dort, wo Frontend und Backend voneinander abhängen
- Der Nachweis entsteht ohne Zusatzaufwand: Pull-Request-Historie und CI-Status fallen ohnehin an
- Grenze: die Tiefe hängt an der Sorgfalt des Reviewers
- Grenze: der Merge ist bisher technisch nicht gesperrt, die Regel beruht auf Disziplin im Team

P3 – Automatisierte Tests

Warum

- Qualitätsziel: funktionale Eignung
- Fehler werden bei jedem Push gefunden, nicht erst im Review oder in Staging
- Bereits funktionierender Code wird mitgeprüft – Rückschritte durch neue Änderungen fallen sofort auf
- Die Prüfung läuft maschinell und damit unabhängig von Aufmerksamkeit und Zeitdruck
- Die Coverage-Grenze verhindert, dass der Testumfang über die Zeit unbemerkt abnimmt

Wer

- Ausführung durch die CI, automatisch
- Behebung von Fehlschlägen durch den Autor der Änderung

Wann

- Auslöser: jeder Push und jeder Pull Request auf dev oder main

Was

- Freigabe: alle Tests erfolgreich, Coverage Backend mindestens 70 Prozent, Frontend mindestens 60 Prozent, keine Lint- oder Typfehler
- Nachweis: Workflow-Log, Coverage-Report als Artefakt

Wie

- Statische Prüfung: Ruff im Backend, vue-tsc im Frontend
- Unit-Tests: pytest-Marker unit, Vitest mit happy-dom
- Integrationstests: Marker integration gegen PostgreSQL 16, Zeitgrenze 60 Sekunden je Test – eine Überschreitung gilt als Fehlschlag
- Coverage ermitteln und mit der Grenze abgleichen

Gut genug als Qualitätsmaßnahme weil:

- Läuft bei jedem Push automatisch – niemand muss daran denken, die Prüfung kann nicht übersprungen werden
- Die drei Stufen finden unterschiedliche Fehler: die statische Prüfung erkennt Tipp- und Typfehler ohne Ausführung, Unit-Tests prüfen einzelne Bausteine, Integrationstests das Zusammenspiel
- Integrationstests laufen gegen eine echte PostgreSQL 16 statt gegen einen Ersatz – dadurch wird auch das tatsächliche Datenbankverhalten geprüft

- Die Zeitgrenze von 60 Sekunden verhindert hängende Tests und macht sichtbar, wenn Tests langsamer werden
- Der Nachweis entsteht automatisch als Workflow-Log und Coverage-Report
- Grenze: Coverage misst nur, wie viel Code ausgeführt wird, nicht ob sinnvoll geprüft wird
- Grenze: Tests finden nur, was jemand vorher als Testfall formuliert hat

P4 – Automatisierte Sicherheitsprüfung

Warum

- Qualitätsziel: Sicherheit
- Ein großer Teil des Codes stammt aus Fremdbibliotheken – Schwachstellen dort wirken wie eigene Fehler
- Neue Schwachstellen werden laufend bekannt, auch ohne Änderung am eigenen Code
- Zugangsdaten im Repository bleiben dauerhaft in der Historie

Wer

- Backend und Frontend, Ausführung durch die CI

Wann

- Jeder Push und jeder Pull Request
- Zusätzlich wöchentlich gegen den unveränderten Stand

Was

- Freigabe: kein offener Befund HIGH oder CRITICAL, keine Geheimnisse im Repository
- Nachweis: SARIF-Befunde im Security-Tab, Prüfergebnis im Workflow-Log
- Ohne erfülltes Kriterium keine Veröffentlichung nach GHCR

Wie

- Abhängigkeiten prüfen: pip-audit im Backend, npm audit auf Stufe high im Frontend
- Trivy prüft Dateisystem und gebautes Image, HIGH oder CRITICAL bricht ab
- Gitleaks bei Pull Requests auf main
- Terraform-Scan der Infrastrukturbeschreibung

Gut genug als Qualitätsmaßnahme weil:

- Der wöchentliche Lauf findet Schwachstellen, die erst nach dem letzten Commit bekannt wurden
- Geprüft werden mehrere Ebenen: Abhängigkeiten, Image, Repository-Inhalt und Infrastruktur
- Geprüft wird das gebaute Image, nicht nur der Quellcode
- Die Veröffentlichung ist an das Ergebnis gekoppelt
- Grenze: gefunden wird nur, was bereits als Schwachstelle bekannt ist
- Grenze: Befunde unterhalb HIGH bleiben folgenlos

P5 – Authentifizierung und Zugriffssteuerung

Warum

- Qualitätsziel: Sicherheit
- Drei Zugangswege bilden auf dieselbe E-Mail-Identität ab: Self-Service-UI über die SSO-Bridge, Moodle über LTI 1.3, Direktzugang über OIDC mit PKCE
- Die Werkzeuge aus P4 decken diesen Bereich nicht ab

- Fehler in der Zugriffssteuerung erzeugen keine Fehlermeldung und werden im normalen Betrieb nicht sichtbar
- Geprüft werden muss das Abweisen unzulässiger Zugriffe – dies wird durch Funktionstests nicht abgedeckt

Wer

- Backend
- Änderungen an Token-Exchange und Impersonation in Keycloak erfordern zusätzlich die Freigabe der QS-Verantwortlichen

Wann

- Auslöser: Änderung an Anmeldung, Token-Verarbeitung, Rollenzuordnung oder Keycloak-Konfiguration
- Zusätzlich manuelle Prüfung vor dem Release

Was

- Freigabe: alle Negativtests bestehen, kein Zugangsweg erlaubt Zugriff ohne geprüfte Identität
- Nachweis: eigene Testdatei, die mit den automatisierten Tests aus P3 läuft, sowie Protokoll der manuellen Prüfung vor dem Release

Wie

- LTI-Signaturprüfung mit Negativfällen: ungültige Signatur, abgelaufen, falscher Aussteller, wiederverwendet
- E-Mail-Kollision zwischen zwei Zugangswegen: die Übernahme eines fremden Kontos muss scheitern
- Auth-Code aus dem Token-Exchange: Lebensdauer begrenzt, zweiter Einlöseversuch scheitert
- PKCE-Ablauf, auch mit nicht passendem Code-Verifier
- Rollentrennung: ein Studierendenkonto erreicht auf keinem Weg eine Dozentenfunktion
- Campus-Lizenzierung wird aus der Anmeldung abgeleitet, nicht aus einer Client-Angabe

Gut genug als Qualitätsmaßnahme weil:

- Geprüft wird mit Negativtests, also das Abweisen unzulässiger Zugriffe – zulässige Abläufe sind bereits durch P3 abgedeckt
- Alle drei Zugangswege werden geprüft, einschließlich der Kollision zwischen ihnen
- Die Negativtests liegen als eigene Testdatei vor und laufen mit den automatisierten Tests aus P3, werden also bei jeder Änderung wiederholt
- Änderungen an Token-Exchange und Impersonation erfordern eine zusätzliche Freigabe, da dort das Risiko höher ist
- Grenze: geprüft werden ausschließlich die formulierten Negativfälle

P6 – Staging-Abnahme und Freigabe nach main

Warum

- Qualitätsziel: funktionale Eignung
- Geprüft wird der Stand vor der Übernahme nach main
- Geprüft wird das bereitgestellte System, nicht der Quellcode – Fehler in Bereitstellung und Konfiguration werden durch eine Codeprüfung nicht erkannt
- Der Health-Check belegt nur Erreichbarkeit, nicht die fachliche Funktion

Wer

- QS-Verantwortliche und Teamlead

Wann

- Auslöser: ein Stand auf dev soll nach main
- Der Staging-Deploy läuft bei jedem Merge nach dev

Was

- Freigabe: Health-Check erfolgreich und Abnahme vollständig abgehakt
- Nachweis: Health-Check-Ergebnis, Discord-Meldung, abgehakte Abnahme im Pull Request nach main
- Kein Merge nach main mit offenen Punkten

Wie

- Der Merge nach dev löst den Staging-Deploy aus: Terraform erzeugt, Ansible konfiguriert
- Automatischer Health-Check, das Ergebnis wird in Discord gemeldet
- Abnahme gegen Staging: Anmeldung über die Self-Service-UI und aus Moodle in beiden Sichten, Katalog lädt und filtert, Deployment startet und erreicht einen laufenden Zustand, Umgebung erreichbar mit der im Template beschriebenen Software, Dokumentation nachgezogen, kein offener Fehler S1 oder S2, kein offener HIGH- oder CRITICAL-Befund
- Bei Abweichung auf dev korrigieren
- Der Pull Request nach main enthält die abgehakte Abnahme

Gut genug als Qualitätsmaßnahme weil:

- Geprüft wird das laufende System auf Staging – Fehler in Bereitstellung und Konfiguration werden durch eine Codeprüfung nicht erkannt
- Automatischer und manueller Teil decken Unterschiedliches ab: der Health-Check die Erreichbarkeit, die Abnahme die fachliche Funktion
- Die Abnahme prüft die gesamte Kette in einem Durchlauf: Anmeldung, Katalog, Deployment, Umgebung
- Das Ergebnis hängt am Pull Request nach main und ist damit nachträglich einsehbar
- Grenze: manuelle Prüfung, der Umfang hängt an der Abnahmeliste

P7 – Production-Deployment und Reproduzierbarkeit

Warum

- Qualitätsziel: Weiterentwicklungseignung (CI/CD)
- Reproduzierbarkeit: die Umgebung wird durch Terraform und Ansible beschrieben und daraus erzeugt
- Nachvollziehbarkeit: Workflow-Läufe und die Images in GHCR dokumentieren, welcher Stand ausgeliefert wurde
- Sicherheit über die Scans aus P4 und TLS, Zuverlässigkeit über die Tests und die getrennte Staging-Umgebung
- Manuelle Änderungen auf der VM führen dazu, dass die Beschreibung nicht mehr dem tatsächlichen Zustand entspricht
- Ein fehlerhaftes Deployment muss rückholbar sein, bevor die Ursache bekannt ist

Wer

- App-Templates, Ausführung durch die CI

Wann

- Auslöser: Merge nach main

Was

- Freigabe: Deployment abgeschlossen und Health-Check der Produktionsumgebung erfolgreich
- Nachweis: Workflow-Log, Image-Digest in GHCR, Terraform-Zustand

Wie

- Die CI baut die Images, durchläuft P4 und veröffentlicht nach GHCR über einen Self-Hosted Runner
- terraform validate und plan, Plan lesen, anschließend apply – unerwartete Löschungen brechen ab
- Ansible konfiguriert die Produktions-VM, manuelle Änderungen auf der VM sind unzulässig
- Health-Check gegen die Produktionsdomain
- Bei Fehlschlag Rollback auf den vorherigen Image-Digest vor der Fehlersuche, anschließend Erfassung nach P14

Gut genug als Qualitätsmaßnahme weil:

- Die Umgebung wird vollständig aus Terraform und Ansible erzeugt, dadurch ist derselbe Stand wiederherstellbar
- Der Terraform-Plan wird vor der Anwendung gelesen, unerwartete Löschungen führen zum Abbruch
- Das Rollback erfolgt vor der Fehlersuche, dadurch ist die Ausfallzeit von der Dauer der Analyse unabhängig
- Der ausgelieferte Stand ist über den Image-Digest dokumentiert
- Der Fehler wird anschließend nach P14 erfasst und bleibt damit nachvollziehbar
- Grenze: der Health-Check prüft Erreichbarkeit, nicht die fachliche Funktion
- Grenze: manuelle Änderungen auf der VM sind untersagt, aber technisch nicht verhindert

P8 – Template-Abnahme

Warum

- Qualitätsziel: funktionale Eignung
- Ein Template beschreibt eine Umgebung, die Lehrende und Studierende unverändert nutzen – Abweichungen wirken unmittelbar in der Lehrveranstaltung
- Ein Fehler im Template betrifft alle daraus erzeugten Umgebungen, nicht einen Einzelfall
- Sicherheitskritische Images erfordern eine Abgrenzung, die vor der Freigabe belegt sein muss
- Lizenzpflichtige Software erfordert eine dokumentierte Lizenzlage

?

Wer

- App-Templates
- Bei sicherheitsrelevanten Images zusätzlich Freigabe der QS-Verantwortlichen

Wann

- Auslöser: ein Template wird neu erstellt oder geändert

Was

- Freigabe: alle Prüfpunkte erfüllt, bei sicherheitsrelevanten Images zusätzlich Freigabe der QS-Verantwortlichen
- Nachweis: abgehakte Prüfpunkte, Packer-Build-Log

Wie

- Die VM startet und ist über den vorgesehenen Weg erreichbar (SSH Port 22, bei Windows RDP Port 3389)
- Die im Template beschriebene Software ist vorhanden und startfähig
- Die beschriebenen Netzwerkeigenschaften sind wirksam – UDP zwischen zwei VMs derselben Umgebung
- Die beschriebenen Daten sind vorhanden – DB-Server erreichbar, Beispieldatensatz geladen

- Quota-Validierung vor dem Deployment: die angeforderten Ressourcen passen in das OpenStack-Kontingent
- Lizenzlage dokumentiert, bei Windows je nach Variante RDS-CAL und Lizenzserver oder VDA beziehungsweise A3/A5
- Wiederholbarkeit: zwei Deployments ergeben dieselbe Umgebung
- Bei sicherheitskritischen Templates zusätzlich belegen: netzseitige Abgrenzung, Zugang auf Teilnehmende der Lehrveranstaltung beschränkt, begrenzte Lebensdauer

Gut genug als Qualitätsmaßnahme weil:

- Geprüft wird die erzeugte VM, nicht die Template-Beschreibung
- Die Prüfpunkte decken die im Template zugesagten Eigenschaften ab: Erreichbarkeit, Software, Netzwerk und Daten
- Die Wiederholbarkeit wird durch zwei Deployments belegt statt angenommen
- Die Quota-Validierung erfolgt vor dem Deployment und verhindert Abbrüche durch ein überschrittenes Kontingent
- Sicherheitskritische Templates erhalten eine zusätzliche Freigabe mit belegter Abgrenzung, Zugangsbeschränkung und begrenzter Lebensdauer
- Die Lizenzlage ist dokumentiert und die Nutzung damit nachweisbar zulässig
- Grenze: geprüft wird gegen die Template-Beschreibung – eine unvollständige Beschreibung bleibt unentdeckt

P9 – Integrationsprüfung Moodle und Identity Provider

Warum

- Qualitätsziel: funktionale Eignung
- Drei Systeme: Moodle, LTI, Keycloak
- Schnittstellen sind durch Unit-Tests nicht prüfbar
- Die drei Systeme werden von unterschiedlichen Stellen und ohne gemeinsamen Releasezyklus geändert
- Fehler betreffen Rollenzuordnung und Kontoanlage und werden im normalen Betrieb nicht sichtbar

Wer

- Backend und Frontend

Wann

- Bei Änderung an LTI-Anbindung, Moodle-Stand oder Keycloak-Konfiguration
- Zusätzlich vor jedem Release

Was

- Freigabe: beide Sichten funktionieren gegen den betriebenen Moodle-Stand
- Nachweis: Protokoll mit Moodle- und LTI-Version

Wie

- Die Schritte werden manuell, in einer Sitzung und in fester Reihenfolge durchgeführt

Gut genug als Qualitätsmaßnahme weil:

- Prüfung gegen den real betriebenen Stand, nicht gegen einen Mock
- Prüfung der korrekten Rollenzuordnung
- Die feste Reihenfolge deckt Wechselwirkungen beim E-Mail-Abgleich auf
- Die Versionsangabe im Protokoll macht nachvollziehbar, gegen welchen Stand freigegeben wurde
- Grenze: manuelle Prüfung, aussagekräftig nur für die geprüften Rollen und Zugangswege

P10 – Last- und Leistungstests

Warum

- Qualitätsziel: Leistungseffizienz
- Leistungsprobleme entstehen schrittweise und werden durch Funktionstests nicht erkannt
- Die Nutzung erfolgt in Lehrveranstaltungen, also durch viele Nutzer gleichzeitig – das Verhalten unter Last unterscheidet sich vom Einzelzugriff
- Ohne regelmäßige Messung ist nicht feststellbar, ob eine Verschlechterung eingetreten ist

Wer

- Backend für die API, Frontend für die Ladezeit

Wann

- Alle zwei Wochen sowie vor jedem Release

Was

- Freigabe: vereinbarte Schwellen eingehalten, kein Einbruch gegenüber der letzten Messung
- Nachweis: Lasttest-Report, Messwerte der Ladezeit

Wie

- Lastszenario gegen Staging: mehrere gleichzeitige Nutzer rufen Katalog und eigene Umgebungen ab, Kursgröße als Bezug
- Antwortzeit und Fehlerrate der API messen, p95 als Bezugsgröße
- Ladezeit der Oberfläche beim ersten Aufruf messen
- Dauer eines Deployments vom Klick bis zur nutzbaren Umgebung messen
- Werte alle zwei Wochen fortschreiben

Gut genug als Qualitätsmaßnahme weil:

- Gemessen wird gegen Staging unter gleichzeitigem Zugriff, nicht im Einzelzugriff
- Die Kursgröße dient als Bezug, das Szenario entspricht damit der tatsächlichen Nutzung
- Das Fortschreiben alle zwei Wochen macht schrittweise Verschlechterungen sichtbar, die eine Einzelmessung nicht zeigt
- p95 statt Mittelwert – damit werden auch langsame Einzelfälle erfasst
- Gemessen wird auch die Deployment-Dauer, also die für Lehrende unmittelbar wahrnehmbare Größe
- Grenze: das Freigabekriterium ist erst prüfbar, wenn die Schwellen beschlossen sind

P11 – Benutzbarkeit und Barrierefreiheit

Warum

- Qualitätsziel: Benutzerfreundlichkeit
- Zielgruppe sind Lehrende ohne Cloud-Vorwissen – die Bedienbarkeit entscheidet über die Nutzbarkeit des Systems
- Bedienprobleme erzeugen keine Fehlermeldung und werden durch Funktionstests nicht erkannt
- Die Zusage „1-Click-Application-Template“ muss überprüfbar sein
- Barrierefreiheit unterliegt im Hochschulumfeld rechtlichen Anforderungen

Wer

- Frontend

Wann

- Alle zwei Wochen sowie vor jedem Release
- Die automatisierte WCAG-Prüfung zusätzlich bei jedem Pull Request

Was

- Freigabe: Aufgabenerfolg mindestens 90 Prozent, keine kritischen Barrierefreiheitsfehler
- Nachweis: Testprotokoll mit Messwerten, Bericht des Prüfwerkzeugs

Wie

- Drei bis fünf Testpersonen, davon mindestens eine Lehrperson ohne Cloud-Vorwissen
- Aufgabenbasierter Test ohne Anleitung, erfasst werden Klickzahl, Zeit, Abbrüche und Zögern
- Gegenprobe zum 1-Click-Application-Template: tatsächliche Klickzahl für ein Deployment
- Designabgleich mit der Self-Service-UI: Farben, Typografie, Abstände, Benennung gleichartiger Bedienelemente
- Automatisierte WCAG-2.1-AA-Prüfung im Pull Request, manueller Tastaturdurchlauf vor dem Release
- Befunde nach P14 einstufen

Gut genug als Qualitätsmaßnahme weil:

- Gemessen wird der Aufgabenerfolg von Testpersonen, nicht die Einschätzung des Teams
- Mindestens eine Testperson entspricht der Zielgruppe ohne Cloud-Vorwissen
- Der Test läuft ohne Anleitung – geprüft wird, ob die Oberfläche ohne Erklärung bedienbar ist
- Die Klickzahl beim Deployment prüft die Zusage „1-Click“ gegen den tatsächlichen Ablauf
- Werkzeug und manueller Durchlauf decken Unterschiedliches ab: das Werkzeug formale Verstöße, der Tastaturdurchlauf die tatsächliche Bedienbarkeit
- Befunde werden nach P14 eingestuft und folgen damit demselben Weg wie andere Fehler
- Grenze: drei bis fünf Testpersonen liefern eine qualitative Aussage, keine statistisch belastbare
- Grenze: der Schwellwert von 90 Prozent ist bisher ein Vorschlag

P12 – Dokumentationspflege

Warum

- Qualitätsziel: Weiterentwicklungseignung
- Bessere Wartbarkeit
- Abweichungen in der Dokumentation werden früher erkannt
- Setup, Ports, Compose-Konfiguration, Deployment und API ändern sich häufig und in kleinen Schritten
- Eine veraltete Dokumentation fällt erst beim nächsten Setup auf

Wer

- Pflege: Autor des Pull Requests
- Periodischer Abgleich: QS-Beauftragte des jeweiligen Teams

Wann

- Ereignisgesteuert: bei Änderung an Setup, Ports, Compose-Konfiguration, Deployment oder API
- Periodisch: Gesamtabgleich alle zwei Wochen

Was

- Freigabe: keine Abweichung zwischen Dokumentation und Ist-Zustand
- Doku-Abschnitt im Pull Request
- Ergebnis des Abgleichs

Wie

- Der Pull Request nennt die betroffene Dokumentation, auch wenn keine betroffen ist
- Codeänderung und Dokumentationspflege erfolgen im selben Pull Request
- Geänderte Ports, Variablen und Dienste werden in der Port- und Variablenübersicht nachgezogen
- Beispielausgaben stammen aus dem laufenden System
- Voraussetzungen der Entwicklungsumgebung werden aktuell gehalten, einschließlich Windows-Besonderheiten
- Alle zwei Wochen Abgleich gegen den Ist-Zustand durch die QS des jeweiligen Teams

Gut genug als Qualitätsmaßnahme weil:

- Die Kopplung an den Pull Request verhindert, dass Dokumentation und Code auseinanderlaufen
- Die Pflege erfolgt unmittelbar, dadurch entsteht kein Wissensverlust
- Der Abgleich alle zwei Wochen findet Abweichungen, die außerhalb eines Pull Requests entstanden sind
- Grenze: geprüft wird die Aktualität der Dokumentation, nicht ihre Verständlichkeit

P13 – Beiträge von KI-Agenten

Warum

- Qualitätsziel: Weiterentwicklungseignung
- Die Coverage-Grenze aus P3 reicht hier nicht: entfernt ein Agent Prüfungen aus bestehenden Tests, bleibt die Abdeckung gleich und die Pipeline erfolgreich – geprüft wird aber weniger
- Ein Agent ändert auch Bereiche, die nicht beauftragt waren
- Ohne Kennzeichnung ist später nicht erkennbar, welcher Code von einem Agenten stammt

Wer

- Der Autor des Pull Requests kennzeichnet ihn und legt den Auftrag vor
- Die QS-Beauftragten des Teams prüfen die Änderungen an den Testdateien

Wann

- Auslöser: ein Pull Request enthält ganz oder teilweise agentengenerierten Code

Was

- Freigabe: keine unbegründete Änderung an Testdateien, keine Änderung außerhalb des Auftrags, Laufprotokoll beigefügt
- Nachweis: Kennzeichnung im Pull Request, Änderungsansicht der Testdateien im Pull Request, Protokoll des Agentenlaufs

Wie

- Vor dem Lauf den Auftrag schriftlich festhalten: Zielzustand, betroffene Module, ausgenommene Bereiche
- Der Agent arbeitet testgetrieben: erst der fehlschlagende Test, dann die Umsetzung
- Im Pull Request kennzeichnen, dass der Code agentengeneriert ist, und nennen, wer den Agenten beauftragt hat
- Alle Änderungen unter tests/ vollständig lesen – eine entfernte Prüfung ohne Begründung führt zur Rückweisung
- Änderungen außerhalb des Auftrags entfernen und in einen eigenen Pull Request überführen
- Laufprotokoll beifügen

Gut genug als Qualitätsmaßnahme weil:

- Geprüft wird gezielt das, was die Pipeline nicht erkennt: Änderungen an bestehenden Tests

- Der schriftliche Auftrag macht feststellbar, was außerhalb des Auftrags liegt
- Eine entfernte Prüfung ohne Begründung führt zur Rückweisung – eine feste Regel, keine Einzelfallabwägung
- Grenze: die Änderungen an Testdateien werden manuell gelesen

P14 – Fehlerbehandlung und Schweregrade

Warum

- Qualitätsziel: funktionale Eignung
- Ohne einheitliche Einstufung wird die Dringlichkeit eines Fehlers im Einzelfall unterschiedlich beurteilt
- Mehrere Prozesse setzen die Schweregrade voraus – P1, P6 und P7 verweisen auf S1 und S2
- Ein reproduzierender Test verhindert, dass derselbe Fehler unbemerkt erneut auftritt

Wer

- QS-Verantwortliche
- Offene Einträge werden im Lead-Sync wiederkehrend geprüft

Wann

- Auslöser: Fehler in Staging, in Production oder durch Nutzer gemeldet

Was

- Freigabe: kein offener Fehler der Stufe S1 oder S2 zum Releasezeitpunkt
- Nachweis: Fehlerregister als Issue-Liste mit Schweregrad

Wie

- Jeder Fehler wird im Register erfasst und einer Stufe zugeordnet
- S1 – Produktion nicht nutzbar, Datenverlust oder Zugriff auf fremde Daten: sofortiger Rollback, Vorrang vor allem anderen
- S2 – Kernfunktion ausgefallen (Anmeldung, Deployment), kein Umweg vorhanden: blockiert das Release
- S3 – Funktion eingeschränkt, Umweg vorhanden: in die laufende Arbeit einplanen
- S4 – Darstellung, Formulierung, Komfort: Sammelbearbeitung, kein Releasehindernis
- Zu jedem S1 und S2 entsteht ein reproduzierender Test, bevor der Fehler behoben wird
- Offene Einträge werden im Lead-Sync wiederkehrend geprüft

Gut genug als Qualitätsmaßnahme weil:

- Die Stufen sind an die Auswirkung gebunden, nicht an eine Einschätzung der Dringlichkeit
- Zu jeder Stufe gehört eine festgelegte Reaktion, die Priorisierung wird nicht im Einzelfall verhandelt
- Der reproduzierende Test entsteht vor der Behebung und belegt, dass der Fehler verstanden und anschließend behoben ist
- Grenze: die Einstufung erfolgt durch eine Person, Grenzfälle bleiben auslegungsbedürftig